FPGA RTL Experiment

Fixed-Point Representation and Quantization — Verilog-2001

Implement fixed-point format conversion, rounding, saturation and overflow detection using synthesizable Verilog.

Deliverables: Verilog RTL module, Verilog testbench, simulation guidance and FPGA implementation workflow.

RTL → simulation → synthesis → hardware validation

1. Experiment objective

Represent real-valued samples using a signed integer and an implied binary-point position.

Convert between input and output formats with different widths and fractional-bit counts.

Round discarded fractional bits instead of always truncating.

Saturate out-of-range results and raise an overflow flag.

Use Verilog-2001 constructs: wire/reg, always @*, parameter and module.

2. Fixed-point representation

Real value = raw signed integer × 2^{-F}

Input with F = 8:

raw = 256

real value = 256 / 2^8 = 1.0

Output with F = 4

Output raw = 16

real value = 16 / 2^4 = 1.0

The represented real value is preserved.

The RTL changes the scaling of the integer; it does not use floating-point hardware.

3. Datapath overview



The output is combinational: it updates when the input or parameterized format changes. overflow is high only when the current result is clamped.

4. Verilog module interface

Inputs

- in_data: signed IN_W-bit input

Parameters

- IN_W, IN_FRAC
- OUT_W, OUT_FRAC

Outputs

- out_data: signed OUT_W-bit result
- overflow: saturation indicator

Default formats

- 16-bit input, 8 fractional bits
- 8-bit output, 4 fractional bits

Use .v file extensions and compile as Verilog-2001, not SystemVerilog.

5. RTL: scaling and rounding

```
// Verilog-2001: Fixed-point conversion with rounding, saturation and overflow
// Defaults: signed input has 8 fractional bits; output has 4 fractional bits.
module fixed_point_quantizer #(
    parameter IN_W     = 16,
    parameter IN_FRAC  = 8,
    parameter OUT_W    = 8,
    parameter OUT_FRAC = 4
) (
    input  wire signed [IN_W-1:0]  in_data,
    output reg  signed [OUT_W-1:0] out_data,
    output reg                     overflow
);

    reg signed [63:0] scaled;
    reg signed [63:0] max_out;
    reg signed [63:0] min_out;
    reg signed [63:0] round_bias;
    integer shift_amt;

    always @* begin
        // Assignment sign-extends in_data because it is signed.
        scaled     = in_data;
        max_out    = (64'sd1 <<< (OUT_W-1)) - 1;
        min_out    = -(64'sd1 <<< (OUT_W-1));
        round_bias = 64'sd0;
        shift_amt  = 0;

        if (OUT_FRAC > IN_FRAC) begin
            shift_amt = OUT_FRAC - IN_FRAC;
            scaled = scaled <<< shift_amt;
        end
        else if (IN_FRAC > OUT_FRAC) begin
            shift_amt = IN_FRAC - OUT_FRAC;
            // Round to nearest; exact ties are rounded away from zero.
```

6. RTL: saturation and overflow

```
                                round_bias = 64'sd1 <<< (shift_amt - 1);
            if (scaled >= 0)
                scaled = (scaled + round_bias) >>> shift_amt;
            else
                scaled = (scaled - round_bias) >>> shift_amt;
        end

        if (scaled > max_out) begin
            out_data = {1'b0, {(OUT_W-1){1'b1}}}; // maximum positive
            overflow = 1'b1;
        end
        else if (scaled < min_out) begin
            out_data = {1'b1, {(OUT_W-1){1'b0}}}; // minimum negative
            overflow = 1'b1;
        end
        else begin
            out_data = scaled[OUT_W-1:0];
            overflow = 1'b0;
        end
    end
endmodule
```

7. Rounding and saturation

Rounding

When reducing fractional bits, add or subtract half of one output LSB before arithmetic right shift.

- Positive: add half-LSB
- Negative: subtract half-LSB
- Ties: away from zero

Saturation for signed 8-bit output

- Maximum = +127
- Minimum = −128
- Above range → +127, overflow = 1
- Below range → −128, overflow = 1
- In range → overflow = 0

8. Testbench

The Verilog testbench applies representative signed inputs and prints raw input, raw output and overflow.

- +256 input raw → +16 output raw (both represent +1.0)
- −256 input raw → −16 output raw (both represent −1.0)
- Large positive/negative values test saturation

The testbench uses reg, wire, initial, delay controls and $display; it does not use SystemVerilog-only string task arguments.

Interpret the output real value as out_data / 2^OUT_FRAC.

9. FPGA implementation workflow

Add fixed_point_quantizer.v and tb_fixed_point_quantizer.v to the project.

Compile and run the testbench in a Verilog-compatible simulator.

Inspect waveforms and verify boundary cases, rounding and overflow.

Synthesize the RTL and review resource utilization and timing.

Apply pin/clock constraints, implement, program the FPGA and compare hardware with simulation.

Keep fixed-point scaling consistent with connected signal-processing modules.

10. Important design notes

This is Verilog-2001 RTL, not SystemVerilog: output variables use reg and combinational logic uses always @*.

The design uses a 64-bit signed intermediate; use sensible parameter widths and verify shifts for your chosen configuration.

Overflow is a current-cycle status output, not a sticky flag.

Run simulation before synthesis; this presentation does not claim the code has been compiled on your specific toolchain.

Possible extensions: pipelining, sticky overflow, selectable rounding mode and automated assertions.

Experiment deliverables

1. fixed_point_quantizer.v — Verilog-2001 RTL
2. tb_fixed_point_quantizer.v — Verilog testbench
3. This presentation — concepts, datapath, verification and FPGA workflow

Next: simulate → inspect waveforms → synthesize → validate on FPGA.